
TIME-SIEVE: EXTRACTING TEMPORAL DYNAMICS VIA INFORMATION BOTTLENECK

Ninghui Feng*
HKUST(GZ)

ninghuifeng@hkust-gz.edu.cn

Songning Lai*
HKUST(GZ)

Deep Interdisciplinary Intelligence Lab (*DI²Lab*)
songninglai@hkust-gz.edu.cn

Jiayu Yang
HKUST(GZ)

Deep Interdisciplinary Intelligence Lab (*DI²Lab*)
jyang729@connect.hkust-gz.edu.cn

Fobao Zhou
HKUST(GZ)

fobaozhou@hkust-gz.edu.cn

Zhenxiao Yin
HKUST(GZ)

zyin368@connect.hkust-gz.edu.cn

Hang Zhao†
HKUST(GZ)

hangzhao@hkust-gz.edu.cn

ABSTRACT

Time series forecasting has become an increasingly popular research area due to its critical applications in various real-world domains such as traffic management, weather prediction, and financial analysis. Despite significant advancements, existing models face notable challenges, including the necessity of manual hyperparameter tuning for different datasets, and difficulty in effectively distinguishing signal from redundant features in data characterized by strong seasonality. These issues hinder the generalization and practical application of time series forecasting models. To solve this issues, this paper propose an innovative time series forecasting model - **TimeSieve** designed to address these challenges. Our approach employs wavelet transforms to preprocess time series data, effectively capturing multi-scale features without the need for additional parameters or manual hyperparameter tuning. Additionally, this paper introduce the information bottleneck theory that filters out redundant features from both detail and approximation coefficients, retaining only the most predictive information. This combination reduces significantly improves the model's accuracy. Extensive experiments demonstrate that our model outperforms existing state-of-the-art methods in most cases, achieving higher predictive accuracy and better generalization. Our results validate the effectiveness of our approach in addressing the key challenges in time series forecasting, paving the way for more reliable and efficient predictive models in practical applications.

1 Introduction

The field of time series forecasting has experienced significant advancements and found widespread applications in various real-world engineering tasks, such as traffic management [1], pandemic spread [2], and financial analysis [3]. From early models like GRU [4] and LSTM [5] to the more recent Transformer-based architectures [6, 7], a plethora of sophisticated models have emerged. For instance, TimesNet [8] and FreTS [9] approach data processing from a frequency domain perspective, while multi-scale models like Pyraformer [10] and Scaleformer [11] leverage intricate multi-scale and cross-dimensional dependency structures to significantly enhance performance on complex forecasting tasks.

*The first two authors contributed equally to this work.

†Correspondence to Hangzhao {hangzhao@hkust-gz.edu.cn}

However, existing models often require the introduction of additional parameters to capture multi-scale features, which increases computational complexity. Moreover, these models necessitate manual tuning of hyperparameters for different datasets, further complicating their application and limiting their generalization across diverse scenarios. More importantly, many models, including Autoformer [7], struggle to effectively distinguish between signal and redundant features when dealing with data characterized by strong seasonality. Redundant features not only interfere with the learning process, leading to increased errors, but also obscures the true patterns within the data, making the predictions unreliable. This issue is particularly pronounced when traditional autocorrelation mechanisms break down due to redundant features interference, significantly diminishing the practical value of these models.

To address these challenges, we propose an innovative time series forecasting model designed to significantly improve predictive accuracy and broaden applicability. First, we employ wavelet transforms to preprocess the time series data using the Wavelet Decomposition Block (WDB) and the Wavelet Reconstruction Block (WRB). Wavelet transforms efficiently capture information from multiple scales, but this process may also introduce redundant features. To address this, we integrate the Information Filtering and Compression Block (IFCB), an information bottleneck module that compresses and extracts the most relevant features by filtering out redundant features from both detail and approximation coefficients. These components work together, significantly enhancing the model’s performance.

Our main contributions can be summarized as follows:

- This paper introduces WDB and WRB that efficiently capture multi-scale features in time series data without requiring manual hyperparameter tuning or the introduction of additional parameters.
- This paper utilizes an information-theoretic approach by incorporating an IFCB to filter out redundant features from the data. This ensures that only the most predictive features are retained, enhancing the model’s accuracy.
- By integrating wavelet transforms for comprehensive feature extraction and the information bottleneck block for redundant features, our model achieves state-of-the-art performance, demonstrating superior predictive accuracy and generalization across diverse scenarios.

2 Related Work

2.1 Time Series Forecasting

In recent years, the field of time series forecasting has advanced rapidly, giving rise to many efficient models. These models predominantly rely on multilayer perceptron (MLP) approaches like MSD-Mixer [12], TSMixer [13], DLinear [14] and FreTs [9], along with Transformer-based techniques such as PatchTST [15], Autoformer, and Informer [16], to perform predictions using sophisticated data transformation and learning mechanisms.

Time series forecasting can be defined as follows: Given a historical sequence of observations $X = \{x_1, x_2, \dots, x_T\}$, where T is the lookback window length, the goal is to predict future values $Y = \{y_{T+1}, y_{T+2}, \dots, y_{T+H}\} = F(X)$, where H is the forecast window length and F is the predictive model mapping the input sequence X to the output sequence Y . However, time series data often exhibit complex, multi-scale characteristics. Conventional feature extraction methods, like standard pooling, require additional parameter adjustments or rely on manually designed pooling sizes, increasing model complexity and limiting flexibility. Wavelet transform, a classical signal processing technique [17], excels in multi-scale analysis. It decomposes signals into different frequency components, aiding in the understanding and prediction of intricate patterns.

We propose a novel approach that combines wavelet transform with modern machine learning techniques for time series forecasting. The wavelet transform preprocesses the input data, decomposing it into detail coefficients and approximation coefficients at different scales. We believe this method effectively captures comprehensive features from the time series data.

2.2 Information Bottleneck Theory

In recent years, with the growing intersection of information theory and deep learning research, many researchers have begun analyzing and optimizing neural networks from the perspective of information theory. In this trend, information bottleneck (IB) theory has gained particular attention [18, 19], finding wide application in emerging fields such as graph neural networks [20], multimodal learning [21], and recently in time series imputation [22]. IB theory helps improve model performance by optimizing information flow and processing [18].

We believe that wavelet transformation can effectively decompose time series data into components that contain valuable information. However, these components also include a significant amount of redundant information. If not properly addressed, this redundant information can interfere with the model’s learning performance and predictive accuracy.

From the perspective of information theory, IB theory provides a mechanism to effectively retain critical information related to target variables while filtering out irrelevant or redundant information in the model’s intermediate representations. This approach enhances the model’s ability to capture key features. Furthermore, it strikes a delicate balance between compressing input information and maintaining predictive accuracy, ensuring that the model’s output remains precise and concise by:

$$\begin{aligned} \max I(y; z) \\ \min I(x; z) \end{aligned} \quad (1)$$

where $I(y; z)$ represents the mutual information between the output y and the intermediate representation z , and $I(x; z)$ represents the mutual information between the input x and the intermediate representation z . Maximizing $I(y; z)$ ensures that the most relevant information for predicting the output is retained, while minimizing $I(x; z)$ helps to compress the input information, filtering out redundant features.

Therefore, this work seeks to apply information-theoretic methods to time series forecasting. Unlike prior applications of the IB theory in deep learning, we do not simply apply it directly. Instead, we integrate it into the overall framework to specifically handle the components obtained from the wavelet transformation. In this way, IB theory can effectively filter out redundant features from these coefficients while retaining key information, ultimately enhancing the model’s predictive performance.

3 TimeSieve

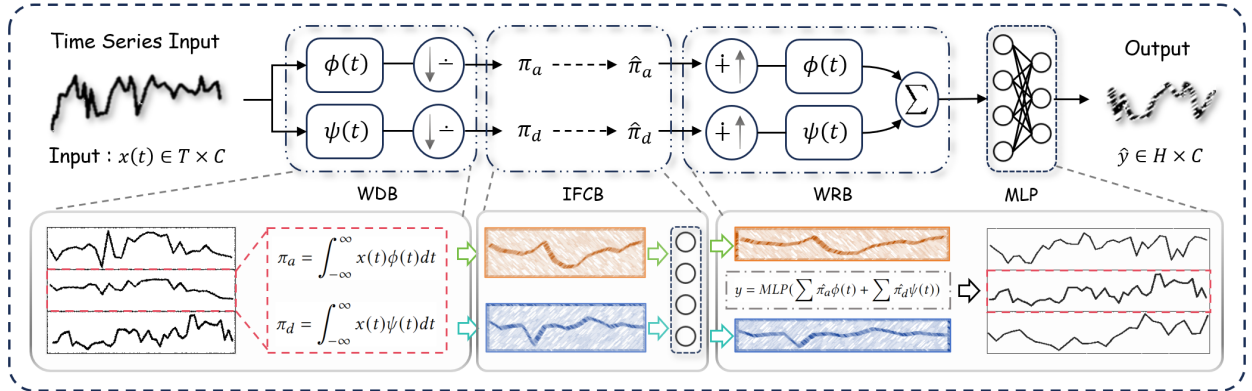


Figure 1: Overview of the TimeSieve Framework. The upper part illustrates the overall architecture of TimeSieve, which consists of WDB, WRB and IFCB. The input time series data, $T \times C$, is decomposed by WDB into coefficients π_a and π_d , which are processed by IFCB. The WRB then reconstructs the data, followed by the predictor generating the corresponding forecast steps. The lower part provides a detailed visualization of the data flow within WDB and IFCB, with further visualization of IFCB available in Figure 2.

In our study, we propose a novel time series forecasting model named **TimeSieve**. This model integrates the Information Filtering and Compression Block (IFCB) with wavelet transform method to enhance the accuracy of time series predictions.

We explain the whole framework first, and we explain WDB, WRB and IFCB in more detail after that.

To effectively capture comprehensive feature information, we employ the Wavelet Decomposition Block (WDB). The WDB decomposes time series data into different frequency components, effectively extracting multi-scale information. Specifically, the wavelet transform can be represented by the following equations:

$$[\pi_a, \pi_d] = WDB(x(t)) \quad (2)$$

where $x(t)$ represents the input, the approximation coefficients π_a represents the low-frequency trends, and the detail coefficients π_d represents the high-frequency details, respectively.

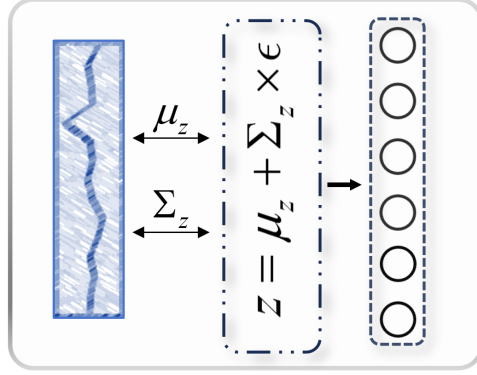


Figure 2: Overview of IFCB: This module consists of a reparameterization step and deep neural networks. Initially, the standard deviation and mean (μ_z, Σ_z) of the data are computed. The reparameterization is then performed using the equation $z = \mu_z + \Sigma_z \times \epsilon$. The input coefficients are processed through the first neural network to obtain μ_z and Σ_z . The reparameterized z is then fed into the second neural network to produce the reconstructed coefficients $\hat{\pi}_i$.

However, the extracted multi-scale information may contain redundant features, which can adversely affect the model's learning performance and predictive accuracy. To address this issue, we introduce the IFCB to filter the information. Additionally, we employ residual connections to ensure completely keeping the valuable information during the filtering process. The IFCB optimizes the information flow, retaining critical information while filtering out irrelevant or redundant information. The IFCB could be shown as following equation:

$$\hat{\pi}_a = IFCB(\pi_a), \hat{\pi}_d = IFCB(\pi_d) \quad (3)$$

where $\hat{\pi}_a, \hat{\pi}_d$ represents the filtered coefficients π_a, π_d .

After filtering with the IFCB, we apply the Wavelet Reconstruction Block (WRB) to reconstruct the processed data back into the time domain. This step ensures that the features at different scales are fully utilized. Finally, this paper use an MLP as the prediction layer to make the final forecast on the processed time series data, specifically adjusting the output to match the desired forecast length. The prediction step is formulated as follows:

$$Y = MLP(WRB(\hat{\pi}_i)), i \in [a, d] \quad (4)$$

The overall architecture of the TimeSieve model is illustrated in Figure 1. The TimeSieve model leverages the combination of wavelet transform and IFCB to effectively handle the multi-scale characteristics present in time series data, thereby improving the model's predictive performance. This approach is motivated by the need to optimize information flow and enhance feature extraction, ensuring that our model can make accurate and reliable predictions across various applications.

The above is an overall description of TimeSieve. We will provide a detailed introduction to WDB, WRB, and IFCB in the following sections.

WDB and WRB. To effectively capture comprehensive feature information, we employ wavelet transformation to decompose time series data inspired by [23, 24], allowing us to analyze the data across multiple scales. Wavelet transformation allows us to break down the data into various frequency components, enabling us to analyze and extract features across multiple temporal scales. This multi-scale analysis is crucial for identifying patterns and trends that occur at different time resolutions.

WDB. For the input time series $x(t) \in \mathbb{R}^{T \times C}$, where T represents the length of the lookback window and C denotes the number of variates, the wavelet transform decomposes the series into approximation coefficients π_a and detail coefficients π_d . The approximation coefficients π_a capture the general trend or smooth component of the series, representing low-frequency information. In contrast, the detail coefficients π_d capture the fine-grained, high-frequency information, highlighting local fluctuations and details within the series. This decomposition can be expressed as:

$$\pi_i = \int x(t)\phi(t)dt, i \in [a, d] \quad (5)$$

where $\phi(t)$ and $\psi(t)$ are the scaling function and wavelet function, respectively, defined as follows:

$$\phi(x) = \sum_{k=0}^{N-1} a_k \phi(2u - k), \psi(x) = \sum_{k=0}^{M-1} b_k \psi(2u - k) \quad (6)$$

In the case of wavelet transform, $\phi(t)$ represents the low-pass filter, capturing the smooth, approximate part of the signal, and $\psi(t)$ represents the high-pass filter, capturing the detailed, high-frequency part of the signal. Here, a_k and b_k are the filter coefficients for the scaling and wavelet functions, respectively, u is the variable over which the functions are defined, and k is the index of the coefficients.

This decomposition allows the model to capture different characteristics of the data at various levels, thereby optimizing the understanding and prediction of time series dynamics, particularly in identifying local features within the series.

IFCB. Motivated by the need to effectively filter out redundant features while retaining critical information, we introduce IFCB to achieve this goal, inspired by the information bottleneck principle [18, 19, 25]. In this section, we process the approximation coefficients π_a and detail coefficients π_d from the wavelet transform of the time series data, for convenience in subsequent explanations, we define π_i and $i \in [a, d]$ to collectively represent π_a and π_d . Additionally, We use $\hat{\pi}_i$ to represent the filtered form(s) of π_i . Initially, we define the input to the IFCB as $\pi_i \in \mathbb{R}^{T/2 \times C}$ and the output to the IFCB as $\hat{\pi}_i \in \mathbb{R}^{T/2 \times C}$.

In our model, both π_i and $\hat{\pi}_i$ are treated as random variables, which given a joint distribution $p(i, \hat{i})$. The mutual information between these two random variables denoted as $I(\pi_i; \hat{\pi}_i)$, quantifies the amount of information shared by π_i and $\hat{\pi}_i$. Mutual information between joint distribution is defined as followed,

$$\begin{aligned} I(\pi_i; \hat{\pi}_i) &= D_{KL}[p(i, \hat{i}) \| p(i)p(\hat{i})] = \sum_{i \in \mathcal{I}, \hat{i} \in \hat{\mathcal{I}}} p(i, \hat{i}) \log \left(\frac{p(i, \hat{i})}{p(i)p(\hat{i})} \right) \\ &= \sum_{i \in \mathcal{I}, \hat{i} \in \hat{\mathcal{I}}} p(i, \hat{i}) \log \left(\frac{p(i|\hat{i})}{p(i)} \right) = H(\pi_i) - H(\pi_i | \hat{\pi}_i) \end{aligned} \quad (7)$$

where $D_{KL}[p \| q]$ is the Kullback-Liebler divergence of the distributions p and q , and $H(P)$ and $H(P|Q)$ are the entropy and conditional entropy of P and Q , respectively.

We define the intermediate hidden layer random variable Z , which follows the Markov chain principles, where $\pi_i \rightarrow Z \rightarrow \hat{\pi}_i$, the mutual information satisfies $I(\pi_i; Z) \geq I(\pi_i; \hat{\pi}_i)$. Although there is an inevitable information loss in the transmission to the middle layer Z , notice that Z retains the most critical information for predicting $\hat{\pi}_i$ while filtering out irrelevant features.

This property makes the definition of objective of IFCB reasonable, which is to maximize the mutual information between Z and $\hat{\pi}_i$ while minimizing the information between Z and π_i :

$$\min(I(\pi_i; Z) - \beta I(Z; \hat{\pi}_i)) \quad (8)$$

where β is a trade-off parameter, which determines the weight of the minimal information constraint in the optimization process.

The optimization is carried out within a deep neural network, which is designed to minimize the information between Z and π_i by adjusting the network weights and biases through backpropagation and suitable regularization techniques.

Then optimization is performed within a deep neural network. Assuming that $p(z|i)$ follows a Gaussian distribution, and the following derivation is established:

$$p(z|i) = \mathcal{N}(\mu(i; \theta_\mu), \Sigma(i; \theta_\Sigma)) = \mathcal{N}(\mu_z, \Sigma_z) \quad (9)$$

where θ_μ and θ_Σ are the parameters of the networks that are optimized during training.

The parameter updating process posing a new challenge due to the stochastic property of the gradient calculations. To address the challenge, the reparameterization trick is introduced. The reparameterization is formulaed by:

$$z = \mu_z + \Sigma_z \times \epsilon \quad (10)$$

where ϵ is sampled from a standard Gaussian distribution.

Given that the study focuses on a regression task, we define the conditional probability $q(\hat{i}|z)$:

$$q(\hat{i}|z) = e^{-\|\hat{i} - D(z; \theta_c)\|} + C \quad (11)$$

where D is a decoder function parameterized by θ_c , and C is a constant. This formulation measures the fit of the model's fitting by capturing the deviation of the predicted output $\hat{i}$ from the output of the decoder.

WRB. After the wavelet transformation, the detail and approximation coefficients are processed through IFCB to preserve the most predictive information. Subsequently, the inverse wavelet transform is applied to reconstruct the time series from the processed coefficients:

$$\hat{x}(t) = \sum \hat{\pi}_a \phi(t) + \sum \hat{\pi}_d \psi(t) \quad (12)$$

where $\hat{\pi}_a$ and $\hat{\pi}_d$ are the filtered approximation and detail coefficients, respectively. Finally, the reconstructed time series is passed through a MLP for final prediction.

Algorithm 1 Pseudo code of TimeSieve

Require: Time series data $x(t) = \{x_1, x_2, \dots, x_T\}$, true labels $Y = \{y_{T+1}, y_{T+2}, \dots, y_{T+H}\}$

Ensure: Predicted output $\hat{Y}$

Initialization: Initialize the model parameters, learning rate η , and total training epochs E

Training:

for $epoch \leftarrow 1$ **to** E **do**

$\pi_a, \pi_d \leftarrow \text{WDB}(X(t))$

for $i \in \{a, d\}$ **do**

$\hat{\pi}_i, \mu_z, \Sigma_z \leftarrow \text{IFCB}(\pi_i)$

end for

$X_{\text{reconstructed}} \leftarrow \text{WRB}(\hat{\pi}_a, \hat{\pi}_d)$

$\hat{Y}_i \leftarrow \text{MLP}(X_{\text{reconstructed}})$

$\mathcal{L}_o \leftarrow \text{LossFunction}(\hat{Y}_i, Y_i)$

for $i \in \{a, d\}$ **do**

$\mathcal{L}_{IB_I} \leftarrow D_{KL}[\mathcal{N}(\mu_{zI}, \Sigma_{zI}) \parallel \mathcal{N}(0, I)] + D_{KL}[p(z_I) \parallel p(z_I|i)]$

end for

$\mathcal{L}_{IB} \leftarrow \mathcal{L}_{IB_A} + \mathcal{L}_{IB_D}$

$\mathcal{L} \leftarrow \mathcal{L}_o + \mathcal{L}_{IB}$

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$

end for

Inference:

$\pi_a, \pi_d \leftarrow \text{WDB}(x(t))$

for $i \in \{a, d\}$ **do**

$\hat{\pi}_i, \mu_z, \Sigma_z \leftarrow \text{IFCB}(\pi_i)$

end for

$X_{\text{reconstructed}} \leftarrow \text{WRB}(\hat{\pi}_a, \hat{\pi}_d)$

$\hat{Y}_i \leftarrow \text{MLP}(X_{\text{reconstructed}})$

return final predictions $\hat{Y}$

Loss Funtion. We now specify the model's loss function. This function is crucial for training the model to minimize prediction errors and optimize the distribution parameters effectively. The loss is composed of the original loss component and the IFCB loss, as shown below:

$$\mathcal{L} = \mathcal{L}_o + \mathcal{L}_{IB} = \mathcal{L}_o + D_{KL}[\mathcal{N}(\mu_z, \Sigma_z) \parallel \mathcal{N}(0, I)] + D_{KL}[p(z) \parallel p(z|i)] \quad (13)$$

where $\mathcal{L}_o$ is the original prediction task loss, typically representing the error in regression predictions, and $\mathcal{L}_{IB}$ is the IFCB loss.

The overall architecture of IFCB is illustrated in Figure 2 and pseudo code of TimeSieve is shown in Algorithm 1.

Table 1: Forecasting results with different forecast lengths $H \in \{48, 96, 144, 192\}$. We set the lookback length $T = 2H$. Red indicates the best result, while Blue refers to the second best.

Models		TimeSieve		Koopa		PatchTST		DLinear		NSTformer		LingTS		Autoformer	
	Metric	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ETTh1	48	0.361	0.341	0.385	0.364	0.375	0.342	0.372	0.342	0.465	0.614	0.406	0.404	0.432	0.678
	96	0.384	0.376	0.411	0.406	0.395	0.377	0.393	0.380	0.498	0.653	0.431	0.435	0.496	0.578
	144	0.397	0.393	0.426	0.424	0.412	0.394	0.401	0.394	0.536	0.602	0.442	0.453	0.521	0.761
	192	0.408	0.402	0.434	0.430	0.437	0.416	0.416	0.408	0.543	0.684	0.457	0.471	0.568	0.598
ETTh2	48	0.291	0.192	0.301	0.241	0.301	0.223	0.307	0.228	0.357	0.339	0.329	0.297	0.414	0.458
	96	0.333	0.302	0.375	0.309	0.350	0.303	0.350	0.295	0.417	0.421	0.375	0.376	0.425	0.490
	144	0.361	0.335	0.368	0.338	0.394	0.345	0.394	0.354	0.489	0.583	0.401	0.420	0.497	0.680
	192	0.382	0.377	0.388	0.353	0.410	0.382	0.410	0.387	0.488	0.454	0.457	0.465	0.512	0.682
ETTm1	48	0.334	0.315	0.336	0.303	0.338	0.289	0.356	0.323	0.395	0.412	0.354	0.348	0.529	0.670
	96	0.335	0.303	0.347	0.307	0.346	0.305	0.347	0.312	0.406	0.492	0.353	0.318	0.482	0.624
	144	0.347	0.322	0.359	0.341	0.362	0.320	0.360	0.326	0.466	0.562	0.367	0.345	0.512	0.683
	192	0.356	0.334	0.294	0.347	0.375	0.344	0.363	0.337	0.454	0.533	0.376	0.361	0.531	0.591
ETTm2	48	0.230	0.147	0.231	0.152	0.232	0.135	0.239	0.148	0.254	0.189	0.247	0.192	0.279	0.241
	96	0.252	0.184	0.254	0.179	0.253	0.172	0.254	0.170	0.307	0.244	0.278	0.265	0.372	0.364
	144	0.272	0.214	0.277	0.216	0.284	0.213	0.276	0.201	0.342	0.288	0.309	0.288	0.408	0.352
	192	0.287	0.223	0.294	0.236	0.294	0.224	0.291	0.220	0.543	0.341	0.327	0.430	0.485	0.892
Exchange	48	0.139	0.045	0.149	0.046	0.145	0.048	0.145	0.046	0.187	0.073	0.159	0.067	0.205	0.124
	96	0.195	0.083	0.211	0.092	0.204	0.090	0.223	0.089	0.294	0.159	0.247	0.168	0.778	0.409
	144	0.241	0.113	0.265	0.141	0.265	0.138	0.256	0.133	0.375	0.292	0.272	0.310	0.680	0.671
	192	0.284	0.180	0.329	0.212	0.298	0.181	0.301	0.182	0.464	0.494	0.354	0.403	0.979	0.544
Electricity	48	0.247	0.155	0.275	0.175	0.251	0.143	0.242	0.158	0.259	0.156	0.287	0.209	0.290	0.198
	96	0.243	0.152	0.285	0.184	0.244	0.144	0.245	0.155	0.277	0.180	0.271	0.181	0.303	0.201
	144	0.243	0.151	0.291	0.192	0.244	0.147	0.247	0.153	0.287	0.191	0.278	0.186	0.320	0.211
	192	0.245	0.152	0.299	0.198	0.241	0.144	0.246	0.155	0.296	0.189	0.291	0.204	0.319	0.216
Weather	48	0.182	0.128	0.171	0.127	0.183	0.137	0.200	0.155	0.218	0.159	0.185	0.136	0.242	0.187
	96	0.199	0.154	0.203	0.155	0.203	0.158	0.223	0.180	0.222	0.161	0.206	0.159	0.283	0.230
	144	0.224	0.166	0.225	0.171	0.226	0.175	0.241	0.192	0.238	0.178	0.227	0.172	0.310	0.307
	192	0.240	0.192	0.241	0.193	0.241	0.197	0.257	0.211	0.254	0.199	0.243	0.196	0.355	0.344

4 Experiments

4.1 Datasets and Baselines

We conduct comprehensive experiments to assess the performance of TimeSieve. For multivariate forecasting, we utilize a variety of real-world benchmarks, including ETT [16], Exchange [26], Electricity [27], and Weather [14]. Notably, instead of using a fixed lookback window length, we follow the approach of Koopa [28] by setting the lookback window length T to twice the forecast window length (i.e., $T = 2H$). This approach is based on the fact that in real-world scenarios, historical observations are readily accessible, and utilizing a larger amount of observed data can enhance the performance of deep models, particularly as the forecast horizon extends [28].

We extensively compare TimeSieve with state-of-the-art deep forecasting models, including Transformer-based models such as PatchTST [15], Nonstationary-Transformer (NSTformer) [6] and Autoformer [7]; MLP-based models such as LightTS [29], DLinear [14] and Koopman operator-based models such as Koopa [28]. See the **Appendix Implementation Details** for a more detailed introduction.

4.2 Comparative Experiment

We present the forecasting results in Table 1, where Red indicates the best result and Blue refers to the second best. Across Table 1, TimeSieve consistently outperformed the baseline models in most scenarios, particularly excelling in datasets with redundant features, such as ETTh1 and ETTh2. In the ETTh1 dataset, TimeSieve achieved a significant reduction in both MAE and MSE at all prediction horizons. At the 48-time step horizon in ETTh1, TimeSieve achieved an MAE of 0.361 and an MSE of 0.341, outperforming Koopa by approximately 6.2% in MAE and 6.3% in MSE. This consistent performance can be attributed to the effective integration of the Wavelet Decomposition Block (WDB) and the Information Filtering and Compression Block (IFCB), which together capture multi-scale features while filtering out redundant features. However, it did not achieve the best results in some experiments with a forecast window of 48.

Table 2: Comparison of sequence decomposition methods with and without wavelet transformations (WDB & WRB and No Wavelet) on Exchange and ETTh2 datasets. We set the lookback length $T = 2H$.

Method		No Wavelet		WDB&WRB	
Metric		MAE↓	MSE↓	MAE↓	MSE↓
Exchange	48	0.151	0.050	0.139	0.045
	96	0.204	0.093	0.195	0.083
	144	0.261	0.202	0.241	0.113
	192	0.312	0.251	0.284	0.180
ETTh2	48	0.299	0.246	0.293	0.193
	96	0.342	0.333	0.337	0.308
	144	0.370	0.374	0.363	0.365
	192	0.389	0.402	0.385	0.387
ETTh1	48	0.339	0.331	0.334	0.315
	96	0.342	0.341	0.335	0.303
	144	0.359	0.347	0.347	0.322
	192	0.360	0.352	0.356	0.334

We believe this may be due to the significant short-term fluctuations in time series data, making it difficult for the model to distinguish meaningful signals from redundant features in shorter forecast windows. In contrast, longer windows (e.g., 144, 192) provide more historical data, enabling the model to better capture long-term dependencies and seasonal patterns, and to differentiate redundant features from short-term features.

Interestingly, the Exchange dataset showed the most significant relative improvement. At the 48-time step horizon, TimeSieve achieved an MAE of 0.139 and an MSE of 0.045, outperforming Koopa by 6.7% in MAE and 2.2% in MSE.

In analysis, we observed an interesting phenomenon where the model’s MSE performance is worse compared to MAE. This can be attributed to MSE’s sensitivity to outliers and large errors, as it squares the errors before averaging them. This sensitivity may impact performance, particularly because the model may filter out significant extreme values. This effect is especially pronounced in the ETTh2 dataset, which is characterized by a higher sampling frequency and an operational environment that may be more susceptible to extreme values. The formulas for calculating MSE and MAE are provided in the **Appendix Metric**.

The varying performance across different datasets and prediction horizons (48, 96, 144, 192 steps) underscores the model’s versatility. TimeSieve consistently demonstrated substantial performance gains at longer horizons (144 and 192 steps). This is particularly evident in the ETTh1 and ETTh2 datasets, where longer-term predictions are crucial for timely decision-making. For example, in the ETTh1 dataset at the 192-step horizon, TimeSieve achieved an MAE of 0.408 and an MSE of 0.402, showcasing substantial improvements over other models such as Koopa (MAE 0.434, MSE 0.430) and PatchTST (MAE 0.437, MSE 0.416). Similarly, in the ETTh2 dataset at the 192-step horizon, the model achieved an MAE of 0.356 and an MSE of 0.334, significantly outperforming competitors in terms of both metrics. The integrated wavelet transform and information bottleneck approach effectively balance feature extraction and redundant feature compression, ensuring a certain level of reliable predictions even when data is limited.

In conclusion, TimeSieve’s ability to achieve state-of-the-art performance on the majority of the datasets tested highlights its superior predictive accuracy and generalization capability.

4.3 Ablation Study

To validate the effectiveness of the modules used in our proposed model, we conducted ablation studies focusing on the wavelet transformation and the IFCB components. Further discussions on the selection of wavelet basis functions and the impact of $\mathcal{L}_{IB}$ loss weights can be found in **Appendix Tables**.

Wavelet Transformation.

In Table 2, we compare the performance of our sequence decomposition methods with and without wavelet transformations (WDB&WRB and No Wavelet) on the Exchange, ETTh2 and ETTh1 datasets. The results demonstrate that incorporating wavelet transformations significantly improves forecasting accuracy, as evidenced by lower MAE and MSE values across all forecasting horizons.

Table 3: Comparison of IFCB performance on Exchange and ETTh2 datasets under different conditions. *OnlyA* applies IFCB only on the approximation coefficients after WDB, *OnlyD* applies IFCB only on the detail coefficients after WRB, and 'TimeSieve' applies IFCB on all coefficients.

Models		TimeSieve		<i>OnlyA</i>		<i>OnlyD</i>	
Metric		MAE↓	MSE↓	MAE↓	MSE↓	MAE↓	MSE↓
Exchange	48	0.139	0.045	0.141	0.048	0.139	0.047
	96	0.195	0.083	0.198	0.084	0.195	0.083
	144	0.241	0.113	0.245	0.125	0.245	0.132
	192	0.284	0.180	0.288	0.215	0.290	0.188
ETTh2	48	0.291	0.192	0.293	0.200	0.293	0.239
	96	0.333	0.302	0.337	0.311	0.338	0.312
	144	0.361	0.335	0.363	0.389	0.369	0.389
	192	0.382	0.377	0.385	0.408	0.385	0.406

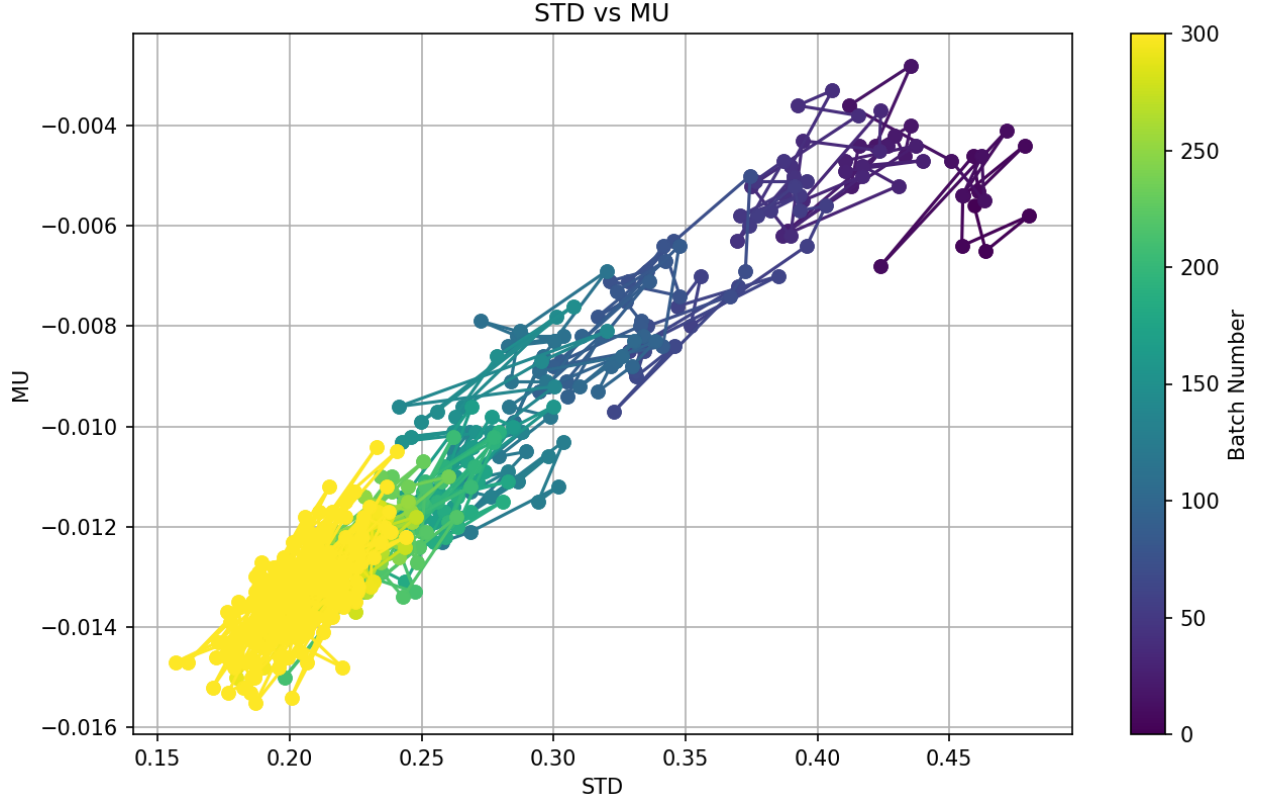


Figure 3: The figure depicts the evolution of the mean (MU) and standard deviation (STD) within the information bottleneck as model iterations progress, with MSE as the metric, for a prediction length of 48 on the Exchange dataset.

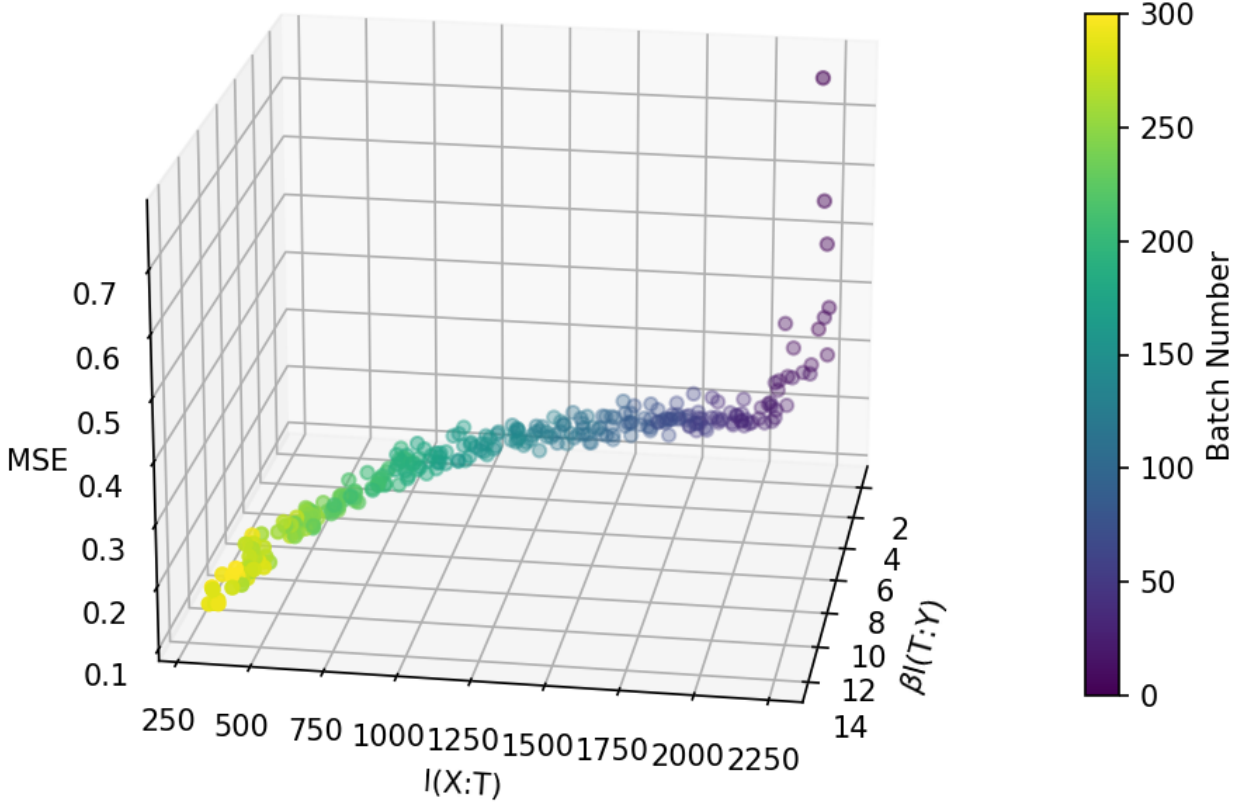
3D Plot of $I(X:T)$ vs $\beta I(T:Y)$ with MSE as Z-axis

Figure 4: The figure illustrates the changes in the mutual information related to the input $I(X;T)$ and the output $\beta I(T;Y)$ throughout model iterations, measured by MSE, for a prediction length of 48 on the Exchange dataset.

By leveraging wavelet transformations, our model can decompose the time series data into different frequency components, effectively compressing redundant features. This process enhances the model’s ability to capture the true underlying trends and patterns, leading to significant improvements in both MAE and MSE. For example, at the 96-step horizon on the Exchange dataset, the MAE decreases from 0.204 to 0.195 and the MSE decreases from 0.093 to 0.083. Similarly, on the ETTh2 dataset, the MAE decreases from 0.342 to 0.337 and the MSE decreases from 0.333 to 0.308.

The results presented in Table 2 validate the effectiveness of using wavelet transformations for time series decomposition. By leveraging the multi-scale nature of wavelets, our method can effectively capture both the coarse and fine-grained structures in the data, leading to more accurate and reliable forecasts.

Information Bottleneck

In Table 3, we present a comparison of our proposed TimeSieve method against two baseline methods, *OnlyA* and *OnlyD*, on the Exchange and ETTh2 datasets. Results show that TimeSieve consistently outperforms the baseline methods across all forecasting horizons. For instance, at a 96-step forecasting horizon on the ETTh1 dataset, TimeSieve achieved a MAE of 0.333 and an MSE of 0.302, which is superior to *OnlyA* (MAE 0.337, MSE 0.311) and *OnlyD* (MAE 0.338, MSE 0.312). However, improvements are more pronounced in the Exchange dataset. In shorter forecast horizons at Exchange, the enhancements are not as apparent, which may be due to the dataset’s richer set of influential features, making it challenging for IFCB to discern effectively at shorter time steps.

Figure 3 presents a scatter plot illustrating the evolution of the standard deviation and mean values in the information bottleneck as the model iterates. In this study, by observing the changes in the mean and standard deviation throughout the training process, we have identified a significant decreasing trend over time, with both parameters eventually converging within a more defined range. The convergence of μ suggests that the model’s latent representations are progressively approaching the central distribution of the data, which aids the model in identifying more representative and universally applicable features across the entire sample space. The reduction in std likely indicates an enhancement in the predictability and certainty of the model’s outputs.

Figure 4 presents a scatter plot of the mutual information values in the objective function as the model iterates. In this study, we observed that during the model training process, the mutual information related to the input $I(X; T)$ gradually decreases with increasing iterations, while the mutual information related to the output $I(T; Y)$ correspondingly increases, alongside a consistent reduction in MSE. This trend validates the effectiveness of the Information Bottleneck (IB) theory in practical applications, demonstrating that the model compresses input information while increasingly retaining information crucial for predicting the output. Specifically, the reduction in $I(X; T)$ reflects the model’s effective filtration of input data during the learning process. Concurrently, the increase in $I(T; Y)$ suggests an enhancement in the model’s capability to extract features closely associated with the task, thereby improving predictive accuracy.

5 Conclusions

In this paper, we presented the TimeSieve framework for time series forecasting, emphasizing the decomposition and reconstruction of time series data through wavelet and information bottleneck techniques. Our approach integrates Wavelet Decomposition Block, Wavelet Reconstruction Block, and Information Fusion and Compression Block to enhance prediction accuracy.

We conducted extensive experiments on multiple benchmark datasets, demonstrating the superiority of TimeSieve in comparison to state-of-the-art methods. The results highlight the model’s capacity to handle varying forecast lengths and different types of time series data.

The proposed TimeSieve framework establishes a foundational approach for time series forecasting across various domains, including finance and climate science. Future work will focus on enhancing the model’s robustness against datasets, alongside exploring extending the model to handle multi-modal time series data.

References

- [1] Ibai Lana, Javier Del Ser, Manuel Velez, and Eleni I. Vlahogianni. Road traffic forecasting: Recent advances and new challenges. *IEEE Intelligent Transportation Systems Magazine*, 10(2):93–109, 2018.
- [2] Naresh Kumar and Seba Susan. Covid-19 pandemic prediction using time series forecasting models. In *2020 11th international conference on computing, communication and networking technologies (ICCCNT)*, pages 1–7. IEEE, 2020.
- [3] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied soft computing*, 90:106181, 2020.
- [4] Rahul Dey and Fathi M. Salem. Gate-variants of gated recurrent unit (gru) neural networks. In *2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 1597–1600, 2017.
- [5] Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. Lstm: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 28(10):2222–2232, 2016.
- [6] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting, 2023.
- [7] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34:22419–22430, 2021.
- [8] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis, 2023.
- [9] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Defu Lian, Ning An, Longbing Cao, and Zhendong Niu. Frequency-domain mlps are more effective learners in time series forecasting, 2023.

- [10] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X. Liu, and Schahram Dustdar. Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *International Conference on Learning Representations*, 2022.
- [11] Amin Shabani, Amir Abdi, Lili Meng, and Tristan Sylvain. Scaleformer: Iterative multi-scale refining transformers for time series forecasting, 2023.
- [12] Shuhan Zhong, Sizhe Song, Weipeng Zhuo, Guanyao Li, Yang Liu, and S. H. Gary Chan. A multi-scale decomposition mlp-mixer for time series analysis, 2024.
- [13] Si-An Chen, Chun-Liang Li, Nate Yoder, Serkan O. Arik, and Tomas Pfister. Tsmixer: An all-mlp architecture for time series forecasting, 2023.
- [14] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting?, 2022.
- [15] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers, 2023.
- [16] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting, 2021.
- [17] Marie Farge. Wavelet transforms and their applications to turbulence. *Annual review of fluid mechanics*, 24(1):395–458, 1992.
- [18] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE information theory workshop (itw)*, pages 1–5. IEEE, 2015.
- [19] Ravid Shwartz-Ziv and Naftali Tishby. Opening the black box of deep neural networks via information. *arXiv preprint arXiv:1703.00810*, 2017.
- [20] Enyan Dai, Limeng Cui, Zhengyang Wang, Xianfeng Tang, Yinghan Wang, Monica Cheng, Bing Yin, and Suhang Wang. A unified framework of graph information bottleneck for robustness and membership privacy, 2023.
- [21] Sijie Mai, Ying Zeng, and Haifeng Hu. Multimodal information bottleneck: Learning minimal sufficient unimodal and multimodal representations. *IEEE Transactions on Multimedia*, 2022.
- [22] MinGyu Choi and Changhee Lee. Conditional information bottleneck approach for time series imputation. In *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Trevor Meyer, Camden Shultz, Najim Dehak, Laureano Moro-Velazquez, and Pedro Irazoqui. Time scale network: A shallow neural network for time series data, 2023.
- [24] Guy P Nason and James L Wei. Leveraging non-decimated wavelet packet features and transformer models for time series forecasting. *arXiv preprint arXiv:2403.08630*, 2024.
- [25] Zichuan Liu, Tianchun Wang, Jimeng Shi, Xu Zheng, Zhuomin Chen, Lei Song, Wenqian Dong, Jayantha Obeysekera, Farhad Shirani, and Dongsheng Luo. Timex++: Learning time-series explanations with information bottleneck, 2024.
- [26] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long-and short-term temporal patterns with deep neural networks. In *The 41st international ACM SIGIR conference on research & development in information retrieval*, pages 95–104, 2018.
- [27] Shengsheng Lin, Weiwei Lin, Wentai Wu, Feiyu Zhao, Ruichao Mo, and Haotong Zhang. Segrnn: Segment recurrent neural network for long-term time series forecasting. *arXiv preprint arXiv:2308.11200*, 2023.
- [28] Yong Liu, Chenyu Li, Jianmin Wang, and Mingsheng Long. Koopa: Learning non-stationary time series dynamics with koopman predictors, 2023.
- [29] David Campos, Miao Zhang, Bin Yang, Tung Kieu, Chenjuan Guo, and Christian S Jensen. Lightts: Lightweight time series classification with adaptive ensemble distillation. *Proceedings of the ACM on Management of Data*, 1(2):1–27, 2023.

A Implementation Details

A.1 Baselines

A.1.1 Koopa [28]

Koopa is a novel forecasting model designed to handle the intrinsic non-stationarity in real-world time series. Developed by researchers at Tsinghua University, Koopa leverages modern Koopman theory to address the complexities associated

with changing temporal distributions. The model decomposes intricate non-stationary series into time-variant and time-invariant components using a Fourier Filter. Subsequently, it employs specialized Koopman Predictors to advance the dynamics of these components independently.

The innovative architecture of Koopa involves stackable blocks that learn hierarchical dynamics, facilitating a modular approach to time series forecasting. Each block integrates Koopman embedding and utilizes Koopman operators as linear representations of implicit transitions. For time-variant dynamics, the model calculates context-aware operators within the temporal neighborhood, ensuring adaptability to localized changes. This method enables Koopa to effectively extend the forecast horizon by incorporating incoming ground truth data.

A.1.2 PatchTST [15]

PatchTST is a Transformer-based model designed for efficient and accurate long-term time series forecasting. Developed by researchers at Princeton University and IBM Research, PatchTST introduces two key innovations: the segmentation of time series into subseries-level patches and a channel-independent approach where each channel corresponds to a single univariate time series. These patches serve as input tokens to the Transformer, enabling the model to retain local semantic information, reduce the computational complexity of attention mechanisms, and effectively utilize longer historical sequences.

The model’s architecture leverages the benefits of patching to enhance locality in the embeddings and minimize the quadratic increase in computation and memory usage. The channel-independent design allows PatchTST to achieve significant improvements in forecasting accuracy by sharing the same embedding and Transformer weights across all channels while processing them independently. PatchTST has demonstrated superior performance in both supervised and self-supervised learning tasks, outperforming state-of-the-art Transformer-based models in various benchmark datasets.

A.1.3 DLinear [14]

DLinear is a simple yet effective model designed to challenge the dominance of Transformer-based models in long-term time series forecasting (LTSF). Developed by researchers at the Chinese University of Hong Kong and the International Digital Economy Academy, DLinear is built on the premise that complex architectures like Transformers may not be necessary for capturing temporal dependencies in time series data.

DLinear operates by decomposing the time series into a trend component and a remainder component. It then applies two separate one-layer linear networks to model these components independently. This direct multi-step (DMS) forecasting approach contrasts with the autoregressive methods commonly used in traditional models, which can suffer from error accumulation over long prediction horizons.

Despite its simplicity, DLinear has demonstrated superior performance across various benchmarks, often surpassing Transformer-based models. Its strengths lie in its ability to capture both short-term and long-term dependencies efficiently, with a low computational cost and high interpretability. DLinear’s success raises important questions about the necessity of using complex Transformer architectures for LTSF and suggests a reevaluation of their application in time series forecasting tasks.

A.1.4 NSTformer [6]

The Nonstationary Transformer (NTSformer) is an advanced framework designed to address the challenges posed by non-stationary time series data in forecasting tasks. Developed by researchers at Tsinghua University, this model builds upon the conventional Transformer architecture, which excels at capturing temporal dependencies through its attention mechanism but struggles with non-stationary data where statistical properties and joint distributions change over time.

To mitigate the limitations of standard Transformers, the Nonstationary Transformer introduces two key modules: Series Stationarization and De-stationary Attention. Series Stationarization employs a normalization strategy to align the statistical properties of input time series, enhancing predictability by ensuring a more stable data distribution. This module normalizes the input series to a common scale, reducing the discrepancies caused by varying means and variances.

However, purely stationarizing the series can lead to the over-stationarization problem, where the model fails to capture crucial temporal dependencies intrinsic to non-stationary data. To address this, the De-stationary Attention module reintegrates the non-stationary characteristics into the attention mechanism. This module learns de-stationary factors that adjust the attention weights, preserving the unique temporal patterns of the original series.

A.1.5 LightTS [29]

LightTS is an innovative framework designed to perform lightweight time series classification, addressing the computational constraints often encountered in resource-limited environments such as edge devices. Developed by researchers from Aalborg University and several collaborating institutions, LightTS utilizes adaptive ensemble distillation to compress large, resource-intensive ensemble models into compact, efficient classifiers without compromising on accuracy.

The core of LightTS involves two primary innovations: adaptive ensemble distillation and Pareto optimal model selection. Adaptive ensemble distillation assigns dynamic weights to different base models, tailoring their contributions based on their classification capabilities, thereby distilling the ensemble knowledge into a single lightweight model. This approach ensures that the most relevant information is captured efficiently, leading to a high-performing student model. Furthermore, LightTS employs a multi-objective Bayesian optimization method to identify Pareto optimal settings that balance model accuracy and size, enabling users to select the most accurate lightweight models that fit within specific storage constraints.

A.1.6 Autoformer [7]

Autoformer is a framework designed to tackle the complexities of long-term time series forecasting. Developed by researchers at Tsinghua University, Autoformer innovatively combines decomposition architecture with an Auto-Correlation mechanism to enhance the model's ability to handle intricate temporal patterns. Traditional Transformer-based models often struggle with long-term dependencies due to the prohibitive computational costs and the obscured nature of temporal dependencies in extended forecasting horizons. Autoformer addresses these challenges by embedding series decomposition blocks within the deep learning architecture, allowing for progressive extraction and refinement of long-term trends and seasonal components.

The Auto-Correlation mechanism is a key innovation of Autoformer, inspired by stochastic process theory. This mechanism leverages the inherent periodicity in time series data to discover dependencies and aggregate similar sub-series efficiently. By replacing the self-attention mechanism with Auto-Correlation, Autoformer achieves a significant reduction in computational complexity, scaling as $O(L \log L)$ for a series of length L . This enables more efficient and accurate long-term forecasting compared to previous models.

A.2 Datasets

Exchange Rate Dataset: The Exchange Rate dataset consists of the daily exchange rates of eight different countries, spanning from 1990 to 2016. This dataset is commonly used to study and forecast currency fluctuations, providing valuable insights for economic planning and financial analysis. The data includes exchange rates for currencies such as USD, GBP, EUR, JPY, and others.

ETT (Electricity Transformer Temperature) Dataset: The ETT dataset includes data collected from electricity transformers, capturing both load and oil temperature measurements at 15-minute intervals over a period from July 2016 to July 2018. This dataset is particularly useful for energy consumption forecasting and transformer health monitoring. The dataset is split into ETTh1, ETTh2, ETTm1, and ETTm2 subsets, representing different transformer units and sampling frequencies (hourly and 15-minutely).

Electricity Consumption Dataset: The Electricity Consumption dataset contains hourly electricity usage data of 321 clients from 2012 to 2014. This dataset is utilized to predict future electricity demand and helps in managing energy resources efficiently. The data captures variations in electricity usage patterns, which are influenced by factors such as weather, economic conditions, and consumer behavior.

Weather dataset The Weather dataset, provided by the Max Planck Institute for Biogeochemistry, comprises detailed meteorological measurements from 2020 to 2021. Recorded every ten minutes, this dataset includes variables such as temperature, humidity, and wind speed. It is essential for high-resolution weather forecasting and climate research, offering insights into microclimatic fluctuations and seasonal patterns. This rich dataset serves as a valuable resource for understanding and predicting weather-related changes over short intervals.

A.3 Definition of Metric

Mean Absolute Error (MAE): The Mean Absolute Error is a metric used to measure the average magnitude of errors between predicted values and actual values. It is defined as follows:

Table 4: Summary of Datasets Used in Time Series Forecasting

Dataset	Time Points	Frequency	Variables
ETTh1	17420	Hour	7
ETTh2	17420	Hour	7
ETTm1	69680	15 Min	7
ETTm2	69680	15 Min	7
Exchange Rate	7588	Day	8
Electricity	26304	Hour	321
Weather	52695	10Min	21

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (14)$$

where y_i represents the actual value, $\hat{y}_i$ represents the predicted value, and n is the number of observations.

Mean Squared Error (MSE): The Mean Squared Error is another metric used to measure the average of the squares of the errors between predicted values and actual values. It is defined as follows:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (15)$$

where y_i represents the actual value, $\hat{y}_i$ represents the predicted value, and n is the number of observations.

A.4 Experimental Setup

Component	Description
GPU	NVIDIA GeForce RTX 3080
CPU	Intel(R) Xeon(R) E5-2686 v4
Epochs	10
Evaluation Metrics	MAE and MSE
Batch Size	32
Optimizer	Adam
Learning Rate	0.0001
Framework	PyTorch 2.0.1
Amount of Memory	32G
CUDA Version	CUDA 11.8

Table 5: Experimental Setup: Summary of Hardware and Software Configurations

B Comparison of of different $\mathcal{L}_{IB}$ loss weights

Table 6: Comparison of different $\mathcal{L}_{IB}$ loss weights on Exchange rate dataset. The lookback length is set to $T = 2H$.

IB Weight Metric	0.000001		0.00001		0.001		0.01		0.1	
	MAE↓	MSE↓	MAE↓	MSE↓	MAE↓	MSE↓	MAE↓	MSE↓	MAE↓	MSE↓
48	0.1394	0.0505	0.1391	0.0451	0.1390	0.0448	0.1389	0.0441	0.1390	0.0440
96	0.1947	0.1102	0.1950	0.0833	0.1946	0.0834	0.1958	0.0857	0.1950	0.0854
144	0.2491	0.1392	0.2414	0.1333	0.2413	0.1133	0.2411	0.1535	0.2403	0.1527
192	0.3008	0.1886	0.2840	0.1802	0.2836	0.1803	0.2834	0.1778	0.2893	0.1804

Table 6 presents the impact of varying information bottleneck ($\mathcal{L}_{IB}$) loss weights on model performance across different forecast horizons on the Exchange rate dataset. Notably, at a 48-step forecast, the lowest MAE and MSE were observed

at IB weights of 0.01 and 0.1, respectively, suggesting that moderate IB weights help minimize predictive errors while maintaining accuracy.

At a 96-step forecast, the optimal MAE appeared at an IB weight of 0.001, while the lowest MSE was at 0.00001, indicating that smaller IB weights can sustain lower errors over longer forecasts.

For longer forecasts (144 and 192 steps), similar trends were observed with the lowest MAE and MSE appearing at adjusted IB weights, indicating that the selection of appropriate IB weights significantly influences model performance. However, when the IB weight is reduced beyond a certain threshold, the performance improvements plateau, suggesting that overly small IB weights might lose their effectiveness.

C Comparison of wavelet basis functions

Table 7: Comparison of different wavelet basis functions on Exchange rate dataset. The lookback length is set to $T = 2H$.

Metric	HAAR		SYM2		DB1	
	MAE↓	MSE↓	MAE↓	MSE↓	MAE↓	MSE↓
48	0.1392	0.0478	0.1420	0.0450	0.1390	0.0448
96	0.1945	0.1024	0.1945	0.0895	0.1946	0.0834
144	0.2446	0.1230	0.2430	0.1326	0.2413	0.1133
192	0.3007	0.2840	0.2914	0.1804	0.2840	0.1803

Table 7 showcases the comparative performance of different wavelet basis functions—HAAR, SYM2, and DB1—on the Exchange rate dataset with a lookback length set to $T = 2H$. The performance metrics used are MAE and MSE, which provide insights into the accuracy and reliability of the predictions at various forecast horizons.

For a 48-step forecast, DB1 demonstrates the lowest MAE and MSE, indicating its superior ability to capture the underlying patterns in the data at shorter horizons. HAAR, while slightly less effective than DB1 in terms of MSE, offers comparable MAE values, suggesting that it remains a robust choice for certain types of financial data.

At a 96-step forecast, HAAR shows the lowest MAE, matching closely with SYM2, but DB1 outperforms both in terms of MSE, highlighting its efficiency in reducing prediction errors over longer horizons.

For 144 and 192-step forecasts, DB1 consistently shows the best performance in both MAE and MSE, underlining its effectiveness in handling longer-term dependencies in the dataset. SYM2, while competitive, tends to exhibit slightly higher errors, particularly in MSE, suggesting that its utility may be more limited under conditions requiring high precision.

Overall, the selection of the appropriate wavelet basis function significantly impacts model performance, with DB1 generally offering the best balance of accuracy and error reduction across the board. This analysis underscores the importance of choosing the right wavelet transform to enhance predictive performance in time series forecasting, especially in the context of financial datasets.